Python-Dictionary

Objective

- Basics of Dictionary
  - Create
  - Insert
  - Update
  - Delete
  - Acess/Iterate
- Built in Dictionary functions
- Dictionary methods
- Dictionary Comprehension

Basics of Dictionary

- Python Dictionary is used to store the data in a key-value pair format.
- It is the mutable data-structure.
- The dictionary is defined into element Keys and values.
- Keys must be a single element.
- Value can be any type such as list, tuple, integer, etc.
- In other words, we can say that a dictionary is the collection of key-value pairs where the value can be any Python object. In contrast, the keys are the immutable Python object, i.e., Numbers, string, or tuple.

Creating Python Dictionary

OBJECT™ TECHNOLOGIES

- Creating a dictionary is as simple as placing items inside curly braces {} separated by commas.
- An item has a key and a corresponding value that is expressed as a pair (key: value).
- While the values can be of any data type and can repeat, keys must be of immutable type (string, number or tuple with immutable elements) and must be unique.
- Separate keys from values with a colon(:), and a pair from another by a comma(,). Finally, put it all in curly braces.
- E.g

  my_dict = {1: ‘Radha Joshi', 2: ‘Gayatri Sathe'}

```
# empty dictionary
my_dict = {}

# dictionary with integer keys
my_dict = {1: 'apple', 2: 'ball'}

# dictionary with mixed keys
my_dict = {'name': 'John', 1: [2, 4, 3]}

# using dict()
my_dict = dict({1:'apple', 2:'ball'})

# from sequence having each item as a pair
my_dict = dict([(1,'apple'), (2,'ball')])
```

As you can see from above, we can also create a dictionary using the built-in dict() function.

Accessing the dictionary values

OBJECT™ TECHNOLOGIES

the values can be accessed in the dictionary by using the keys as keys are unique in the dictionary.

```
Employee = {"Name": "John", "Age": 29, "salary":25000,
"Company":"GOOGLE“ }
print(type(Employee))
print("printing Employee data .... ")
print("Name : %s" %Employee["Name"])
print("Age : %d" %Employee["Age"])
print("Salary : %d" %Employee["salary"])
print("Company : %s" %Employee["Company"])
```

Output:

```
<class 'dict'>
printing Employee data ....
Name : John
Age : 29
Salary : 25000
Company : GOOGLE
```

Declaring one key more than once

```
mydict2={1:2,1:3,1:4,2:4}
print(mydict2)
```

Output:

```
{1: 4, 2: 4}
```

As you can see, 1:2 was replaced by 1:3, which was then replaced by 1:4. This shows us that a dictionary cannot contain the same key twice.

Adding dictionary values

- The dictionary is a mutable data type, and its values can be updated by using the specific keys.
- The value can be updated along with key Dict[key] = value.
- The update() method is also used to update an existing value.
- Note: If the key-value already present in the dictionary, the value gets updated. Otherwise, the new keys added in the dictionary.

www.object.co.in

Example

```
Dict = {}   # Creating an empty Dictionary
print("Empty Dictionary: ")
print(Dict)
```

#**Output**:

```
{}
```

```
# Adding elements to dictionary one at a time
Dict[0] = 'Peter'
Dict[2] = 'Joseph'
Dict[3] = 'Ricky'
print(" Dictionary after adding 3 elements: ")
print(Dict)
```

#**Output**:

```
{0: 'Peter', 2: 'Joseph', 3: 'Ricky'}
```


www.object.co.in

```
# Adding set of values  with a single Key
# The Emp_ages doesn't exist to dictionary
Dict['Emp_ages'] = 20, 33, 24
print(" Dictionary after adding 3 elements: ")
```

#**Output**:

```
print(Dict)   {0: 'Peter', 2: 'Joseph', 3: 'Ricky', 'Emp_ages': (20, 33, 24)}
```

```
# Updating existing Key's Value
Dict[3] = 'Welcome'
print(" Updated key value: ")
print(Dict)
```

#**Output**:

```
{0: 'Peter', 2: 'Joseph', 3: 'Welcome', 'Emp_ages': (20, 33, 24)}
```

Removing elements from Dictionary

OBJECT™ TECHNOLOGIES

- We can remove a particular item in a dictionary by using the pop() method. This method removes an item with the provided key and returns the value.
- The popitem() method can be used to remove and return an arbitrary (key, value) item pair from the dictionary.
- All the items can be removed at once, using the clear() method.
- We can also use the del keyword to remove individual items or the entire dictionary itself.

Removing elements from a dictionary

```
# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# remove a particular item, returns its value
print(squares.pop(4))
#Output: 16

print(squares)
#Output: {1: 1, 2: 4, 3: 9, 5: 25}

# remove an arbitrary item, return (key,value)
print(squares.popitem())
#Output: (5, 25)

print(squares)
#Output: {1: 1, 2: 4, 3: 9}
```


www.object.co.in

```
# remove all items
squares.clear()
print(squares)
#Output:{}

# delete the dictionary itself
del squares

# Throws Error
print(squares)
#Output:
Traceback (most recent call last):
  File "<string>", line 30, in <module>
    print(squares)
NameError: name 'squares' is not defined
```

Deleting a single key-value pair

- To delete just one key-value pair, use the keyword 'del' with the key of the pair to delete.

```
dict4={1:1,2:2,3:3,4:4}
del dict4[2]
print(dict4)
```

Output:

{1: 1, 3: 3, 4: 4}

www.object.co.in

Iterating Dictionary

- A dictionary can be iterated using for loop as given below.

Example 1

```
# for loop to print all the keys of a dictionary
Employee = {"Name": "John", "Age": 29, "salary":25000,
"Company":"GOOGLE"}
for x in Employee:
    print(x)
```

Output:

```
Name
Age
salary
Company
```


www.object.co.in

Example 2

```
#for loop to print all the values of the dictionary
Employee = {"Name": "John", "Age": 29, "salary":25000,
"Company":"GOOGLE"}
for x in Employee:
   print(Employee[x])
```

Output:

John
29
25000
GOOGLE

Example – 3

```
#for loop to print the values of the dictionary by using values() method.
Employee = {"Name": "John", "Age": 29, "salary":25000, "Company":"GOOGLE"}
for x in Employee.values():
    print(x)
```

Output:

```
John
29
25000
GOOGLE
```

Example 4

```
#for loop to print the items of the dictionary by using items()
method.
Employee = {"Name": "John", "Age": 29, "salary":25000,
"Company":"GOOGLE"}
for x in Employee.items():
    print(x)
```

Output:

```
('Name', 'John')
('Age', 29)
('salary', 25000)
('Company', 'GOOGLE')
```

Example 5

```
#for loop to print the items of the dictionary by using items()
method with key and value.
Employee = {"Name": "John", "Age": 29, "salary":25000,
"Company":"GOOGLE"}
for x,y in Employee.items():
    print(x,y)
```

Output:

```
Name John
Age 29
salary 25000
Company GOOGLE
```

Built in Dictionary functions

1. **len(dict)**

   Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

2. **str(dict)**

   Produces a printable string representation of a dictionary

3. **dic()**

   The dict() function creates a dictionary.

Dictionary methods

1. **dict.clear()**
   Removes all elements of dictionary dict

2. **dict.copy()**
   Returns a shallow copy of dictionary dict

3. **dict.fromkeys()**
   Create a new dictionary with keys from seq and values set to value.

4. **dict.get(key, default=None)**
   For key, returns value or default if key not in dictionary

5. **dict.has_key(key)**
   Returns true if key in dictionary dict, false otherwise

www.object.co.in

6. **dict.items()**
   Returns a list of dict's (key, value) tuple pairs

7. **dict.keys()**
   Returns list of dictionary dict's keys

8. **dict.setdefault(key, default=None)**
   Similar to get(), but will set dict[key]=default if key is not already in dict

9. **dict.update(dict2)**
   Adds dictionary dict2's key-values pairs to dict

10. **dict.values()**
    Returns list of dictionary dict's values

Dictionary Comprehension

- Dictionary comprehension is an elegant and concise way to create a new dictionary from an iterable in Python.
- Dictionary comprehension consists of an expression pair (key: value) followed by a for statement inside curly braces {}.

Here is an example to make a dictionary with each item being a pair of a number and its square.

```
# Dictionary Comprehension
squares = {x: x*x for x in range(6)}
print(squares)
```

Output:

{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

www.object.co.in

This code is equivalent

```
tosquares = {}
for x in range(6):
    squares[x] = x*x
print(squares)
```

Output:

{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Summary

- Basics of Dictionary
  - Create
  - Insert
  - Update
  - Delete
  - Acess/Iterate
- Built in Dictionary functions
- Dictionary methods
- Dictionary Comprehension

www.object.co.in

Thank You